Lab 1 — Introduction to SQL Server Management Studio (SSMS)

Introduction to Editor (SQL Server Management Studio)
Installation Steps

Lab 2 — Perform SQL Commands for Creating Tables and Inserting Data (CREATE, INSERT)

Database Name: Branch_DIV_Rollno (Example: CSE_3A_101 or BSC_HONS_101)
Note: Create all the tables under above database.

Create following tables and insert the data into tables using query as shown below.

DEPOSIT	
COLUMN_NAME	**DATATYPE**
ACTNO	INT
CNAME	VARCHAR(50)
BNAME	VARCHAR(50)
AMOUNT	DECIMAL(8,2)
ADATE	DATETIME

ACTNO	CNAME	BNAME	AMOUNT	ADATE
101	MEET	MAVDI	10000	1-3-2025
102	JAY	MADHAPAR	5000	4-1-2026
103	RAHUL	BEDI	3500	7-1-2026
104	RIYA	MAVDI	1200	7-6-2025
105	MANSI	KKV HALL	3000	2-3-2024
106	DIYA	MADHAPAR	2000	1-3-2025
107	MIRAL	BEDI	1000	5-9-2025
108	UDAY	UMIYA CHOWK	5000	2-7-2025
109	CHARMI	SHITAL PARK	7000	2-8-2026
110	BHAVIN	RING ROAD	8000	2-2-2025
111	BANSI	NULL	9000	1-1-2025

STUDENT	
COLUMN_NAME	**DATATYPE**
STDID	INT
SNAME	VARCHAR(50)
CITY	VARCHAR(50)
SPI	DECIMAL(4,2)
BRANCH	VARCHAR(50)

STDID	SNAME	CITY	SPI	BRANCH
101	HETVI	RAJKOT	7.40	COMPUTER
102	RAJ	MORBI	9.50	MECHANICAL
103	VISHAL	RAJKOT	9.00	CIVIL
104	DEEP	SURAT	8.80	COMPUTER
105	DHARMIK	BARODA	8.80	CHEMICAL
106	KRUNAL	VAPI	9.00	CIVIL
107	RIYA	NAVSARI	5.50	COMPUTER
108	VRUNDA	KUTCH	7.60	ELECTRICAL
109	SMAIR	JAMNAGAR	6.80	EC
110	PARAG	SURAT	7.00	CHEMICAL
111	HARSH	RAJKOT	4.00	NULL

EMPLOYEE							
EID	FIRSTNAME	LASTNAME	DEPARTMENT	SALARY	CITY	GENDER	JOININGYEAR
101	HETVI	PATEL	ADMIN	12000.00	RAJKOT	FEMALE	2026
102	RAJ	MEHTA	IT	14000.00	AHMEDABAD	MALE	2022
103	VISHAL	SHARMA	HR	15000.00	BARODA	MALE	2020
104	DEEP	PATEL	ADMIN	12500.00	RAJKOT	MALE	2026
105	DHAVAL	SHAH	IT	14000.00	JAMNAGAR	MALE	2024
106	RIYA	KAUR	IT	5000.00	AHMEDABAD	FEMALE	2024
107	PARAG	PANDYA	HR	7000.00	RAJKOT	MALE	2025
108	VRUNDA	VYAS	SERVER	10000.00	BARODA	FEMALE	2022
109	MEHUL	SINGH	HR	12000.00	MORBI	MALE	2020
110	MUBIN	PARMAR	TRANSPORT	12000.00	SURAT	MALE	2021
111	MAYANK	PUROHIT	ACCOUNT	13000.00	NULL	MALE	2020

Lab 3	**Perform SQL SELECT Queries Using Operators and Conditions**

From the table STUDENT perform the following queries:

Part – A:

1. Retrieve all data from table STUDENT.
2. Display Student Name and City from STUDENT.
3. Display student details of all students who belongs to COMPUTER branch.
4. Display names of students whose ID is less than 105 from STUDENT table.
5. Give Student Name, City and SPI of student whose SPI is greater than 6.50.
6. Give name of Student whose branch is COMPUTER and SPI is greater than 8.00.
7. Give names of students whose ID is greater than 103 and belongs to Rajkot city.
8. Display names of students who belong to either 'RAJKOT' or 'SURAT' city (USE OR & IN).
9. Display names of students with branch whose SPI is greater than 8.0 and ID is less than 105.
10. Find all students whose SPI is greater than or equal to 7.0 and less than or equal to 9.0 (USE AND & BETWEEN).
11. Find all students who do not belong to 'COMPUTER' branch.
12. Display Student ID, Name & SPI of students who belong to 'COMPUTER', 'CIVIL' or 'CHEMICAL' branch and ID is less than 104.
13. Display all student IDs and names who do not belong to 'COMPUTER' or 'CIVIL' branch (USE NOT IN).
14. Display all student names other than 'DEEP' from STUDENT table (USE NOT, <>, !=).
15. Display student names whose branch is not available (NULL) in STUDENT table.
16. Retrieve all unique branches name from STUDENT table.
17. Retrieve first 50% records from STUDENT table.
18. Retrieve first five student IDs from STUDENT table.

Part – B:

19. Display all the details of first five students from STUDENT table.
20. Display all the details of first three students whose SPI is greater than 8.0.
21. Display Student ID, Name of first five students whose branch does not belong to 'COMPUTER' branch.
22. Select all details with student IDs not in the range 105 to 109.
23. Select all records from STUDENT where SPI is greater than 7.0 and less than or equal to 9.0, and student ID is between 102 and 108.

Part – C:

24. Display all details of students who have SPI more than 8.5 without using * from STUDENT table.
25. Retrieve names of students whose city is 'RAJKOT' and SPI is less than 8.00.
26. Retrieve records from STUDENT table where SPI is greater than 8.0 and student ID is less than 105.
27. Retrieve records from STUDENT table where SPI is greater than 7.5 and student ID is between 100 and 110 and city is 'RAJKOT' or 'SURAT'.
28. Display details of students who belong to 'CIVIL' or 'MECHANICAL' branch and SPI is greater than 8.0.

Lab 4	**Perform SQL Queries for Update Data (UPDATE)**

From the table STUDENT perform the following queries:

Part – A:

1. Update SPI of all students from 7.00 to 8.00.
2. Change city of HETVI from RAJKOT to AHMEDABAD.
3. Update SPI of DEEP to 9.20 and city to VADODARA.
4. Update SPI of DHARMIK to 8.50.
5. Update branch name from COMPUTER to IT.
6. Update branch of RAJ to AUTOMOBILE.
7. Update SPI to 7.50 where STDID is between 103 and 107.
8. Update city of PARAG to MUMBAI.
9. Update SPI of RIYA to 6.00.
10. Update SPI of SMAIR to 7.20 and branch to ELECTRICAL.

Part – B:

11. Give 10% increment in SPI.
12. Increase SPI by 20% for all students.
13. Increase SPI by 0.50 in all records.
14. Update branch to 'EC' and SPI to 8.00 and city to Surat where SNAME is KRUNAL.
15. Update city to 'RAJKOT' and SPI to 7.00 where branch is CIVIL and stdid is less than 105.

Part – C:

16. Update SPI of student with stdid 110 to NULL.
17. Update branch of VISHAL to NULL.
18. Display names of students whose SPI is NULL.
19. Display students who have branch assigned.
20. Update student with stdid 108 to name DARSHAN, branch COMPUTER, and SPI 8.50.
21. Update city to SURAT where SPI is less than 7.00.
22. Update city to NULL and branch to MECHANICAL where stdid is 109.

Lab 5	**Perform SQL Queries for ALTER, RENAME, DELETE, TRUNCATE, and DROP Commands**

From the table DEPOSIT perform the following queries:

Part – A:

1. Add column state varchar(20).
2. Add two more columns city varchar(20) and pincode int.
3. Change the size of cname column from varchar(50) to varchar(35).
4. Change the data type of amount from decimal to int.
5. Delete column city from the DEPOSIT table.
6. Rename column actno to ano.
7. Rename column bname to branch_name.
8. Rename table DEPOSIT to DEPOSIT_DETAIL.
9. Add column ifsc_code varchar(15).
10. Change the size of bname column from varchar(50) to varchar(30).

Part – B:

11. Rename column adate to aopendate.
12. Delete column aopendate from DEPOSIT_DETAIL table.
13. Rename column cname to customer_name.
14. Add column country varchar(20).
15. Add column account_type varchar(15).

Part – C:

16. Change data type of pincode from int to bigint.
17. Delete column account_type.
18. Rename column amount to balance.
19. Add column status varchar(10).
20. Change table name deposit_detail to bank_deposit.

From the table DEPOSIT perform the following queries:
Part – A:

1. Delete all the records having amount less than or equal to 3000.
2. Delete all the accounts of 'BEDI' branch customer.
3. Delete all the accounts having account number greater than 102 and less than 109.
4. Delete all the accounts whose branch is 'BEDI' or 'MADHAPAR'.
5. Delete all the accounts details where amount is 8000 and account open after 1-1-2025;
6. Delete all the accounts whose account branch is NULL.
7. Delete all the accounts details where amount is 7000 and name is CHARMI and branch is SHITAL PARK.
8. Delete all the remaining records using DELETE command.
9. Delete all the records of DEPOSIT table. (Use TRUNCATE)
10. Remove DEPOSIT table. (Use DROP)

From the table STUDENT perform the following queries:
Part – B:

11. Delete all the students whose stdid is greater than 105.
12. Delete the records whose branch is NULL and sname is not NULL.
13. Delete the records whose SPI is less than 9 and city is RAJKOT.
14. Delete the records whose branch name is not empty.
15. Delete all the records of STUDENT table. (Use TRUNCATE)

Lab 6 | Perform SQL Queries for Data Copy (SELECT INTO)

From the table DEPOSIT perform the following queries:
Part – A:

1. Copy all records from DEPOSIT where AMOUNT > 3000 into HIGH_AMOUNT.
2. Copy only CNAME and AMOUNT from DEPOSIT where BNAME = 'MAVDI' into MAVDI_CUSTOMERS.
3. Copy records of DEPOSIT where ADATE > '2025-01-01' into RECENT_DEPOSITS.
4. Copy distinct BNAME from DEPOSIT into BRANCH_LIST.
5. Copy top 5 records from DEPOSIT into TOP_DEPOSITS.
6. Copy records where AMOUNT between 2000 and 6000 into MID_RANGE.
7. Copy distinct branch names from DEPOSIT into UNIQUE_BRANCH.
8. Copy records with NULL branch into NO_BRANCH_ASSIGNED.
9. Copy all records and rename AMOUNT as BALANCE into DEPOSIT_COPY.
10. Copy records where BNAME in ('MAVDI','BEDI') into SELECTED_BRANCH.

From the table Student perform the following queries:
Part – B:

11. Create a new table STUDENT_BACKUP from STUDENT without copying any data.
12. Copy SNAME and CITY where BRANCH = 'COMPUTER' into CS_STUDENTS.
13. Copy top 3 students based on SPI into TOPPER_LIST.
14. Copy distinct CITY from STUDENT into CITY_LIST.
15. Copy records where STDID between 103 and 108 into MID_STUDENTS.

	From the table Student perform the following queries: **Part – C:** 16. Copy records with NULL branch into NULL_BRANCH_STUDENTS. 17. Copy all STUDENT records and rename SPI as PERFORMANCE into STUDENT_COPY. 18. Copy records where CITY in ('RAJKOT','SURAT') into CITY_WISE. 19. Copy students where BRANCH <> 'CIVIL' into NON_CIVIL_STUDENTS. 20. Copy selected columns (SNAME, CITY) from STUDENT table into a new table.
Lab 7	**Perform SQL Pattern Searching Using LIKE Operator**
	From the tables EMPLOYEE perform the following queries: **Part – A:** 1. Display employees detail whose FIRSTNAME starts with 'H'. 2. Display employees detail whose FIRSTNAME consists of exactly 5 characters. 3. Display employees detail whose CITY ends with 'T' and has 6 characters. 4. Display employees detail whose LASTNAME ends with 'EL'. 5. Display employees detail whose FIRSTNAME starts with 'R' and ends with 'A'. 6. Display employees detail whose FIRSTNAME starts with 'V' and third character is 'S'. 7. Display employees detail whose CITY is NULL and FIRSTNAME has 6 characters. 8. Display employees detail whose FIRSTNAME contains 'AR'. 9. Display employees detail whose CITY starts with 'R' or 'B'. 10. Display employees detail whose DEPARTMENT is NOT NULL. 11. Display employees detail whose FIRSTNAME starts from alphabet A to H. 12. Display employees detail whose second character of FIRSTNAME is a vowel. 13. Display employees detail whose FIRSTNAME length ≥ 5. 14. Display employees detail whose LASTNAME starts with 'PA'. 15. Display employees detail whose CITY does not start with 'B'. 16. Display employees whose second character of FIRSTNAME is a not vowel. 17. Display employees whose JOINING YEAR last digit is 4 or 6. 18. Display employees detail whose FIRSTNAME starts with 'H', ends with 'I', and CITY contains 'RA'. 19. Display employees detail whose FIRSTNAME contains 'A', CITY ends with 'D', and DEPARTMENT is NOT NULL. 20. Display employees whose second and third characters of FIRSTNAME are vowels and CITY starts with 'R'. **Part – B:** 21. Display employees whose CITY contains 'RA' and salary less than 13000 and joining year last digit is 6. 22. Display employees whose SALARY between 10000 and 15000 and CITY name contains 'KO' and FIRSTNAME start with H. 23. Display employees whose FIRSTNAME starts with 'A' or 'D' and SALARY greater than 12000. 24. Display employees whose CITY contains 'N' and SALARY less than 15000. 25. Display employees whose FIRSTNAME length = 6 and CITY ends with 'AR'. **Part – C:** 26. Display employees whose FIRSTNAME ends with a vowel, department name start with vowel, and SALARY is between 10000 and 15000. 27. Display employees whose LASTNAME contains 'A' at least twice, gender is male, and SALARY is not equal to 14000.

	28. Display employees whose FIRSTNAME second character is vowel and LASTNAME ends with 'R' and SALARY less than 12000. 29. Display employees whose CITY is NOT NULL and FIRSTNAME does not start with vowel and DEPARTMENT not in ('HR', 'IT'). 30. Display employees whose CITY is not NULL, FIRSTNAME ends with vowels, and DEPARTMENT is neither 'HR' nor 'IT'.
Lab 8	**Perform SQL Queries Using Aggregate Functions with GROUP BY Clause (Without HAVING)**
	From the tables EMPLOYEE perform the following queries **Part – A:** 1. Display the Highest, Lowest Salary and Label the columns Maximum, Minimum respectively. 2. Display Total, and Average salary of all employees. Label the columns Total_Sal and Average_Sal, respectively. 3. Find total number of employees of EMPLOYEE table. 4. Find highest salary from Rajkot city. 5. Give maximum salary from IT department. 6. Count employee department is HR. 7. Display average salary of Admin department. 8. Display total salary of HR department. 9. Count total number of cities of employee without duplication. 10. Count unique departments. 11. Display minimum salary of employee who belongs to Ahmedabad. 12. Find city wise highest salary. 13. Find department wise lowest salary. 14. Display minimum salary in each city. 15. Display average salary of employees from Surat. 16. Display total salary of female employees. 17. Count number of male employees. 18. Display city with the total number of employees belonging to each city. 19. Count number of employees in each city where gender is MALE. 20. Display maximum salary in each department where city is not Ahmedabad. **Part – B:** 21. Display minimum salary in each city where gender is FEMALE. 22. Give total salary of each department of EMPLOYEE table. 23. Give average salary of each department of EMPLOYEE table without displaying the respective department name. 24. Count the number of employees for each department in every city. 25. Calculate the total salary distributed to male and female employees. **Part – C:** 26. Give city wise maximum and minimum salary of female employees. 27. Calculate department, city, and gender wise average salary. 28. Display the difference between the highest and lowest salaries. Label the column DIFFERENCE. 29. Display sum of salaries of department wise where department name consist 5 letter. 30. Find the Maximum salary department & city wise in which city name starts with 'R'.

Lab 9	**Perform SQL Queries Using GROUP BY with HAVING Clause and ORDER BY**
	From the tables EMPLOYE perform the following queries **Part – A:** 1. Display cities where total salary of employees greater than 20000. 2. Display departments having average salary greater than 12000. 3. Display departments having total salary greater than 20000. 4. Display departments having number of employees greater than 2. 5. Display cities where minimum salary less than 7000. 6. Display cities where average salary less than 12000. 7. Display departments where maximum salary greater than 14000. 8. Display cities where total salary greater than equal to 30000. 9. Display departments having number of employees equal to 2. 10. Display cities having number of female employees greater than equal to 1. 11. Display departments where minimum salary of male employees greater than 7000. 12. Display cities where maximum salary of female employees less than 13000. 13. Display departments where average salary greater than 10000 and less than 14000. 14. Display cities where number of employees joined before 2023 greater than 1. 15. Display cities where total salary of male employees greater than 15000, ordered by total salary. 16. Display departments where maximum salary greater than 13000, ordered by max salary. 17. Display cities where total salary of male employees greater than 15000. 18. Display departments where employees joined after 2022 and count greater than 1. 19. Display departments where average salary of female employees greater than 8000. 20. Display departments having total salary greater than 20000 and less than 40000. **Part – B:** 21. Display departments having total salary of employees joined after 2021 greater than 20000. 22. Display cities where average salary of employees joined after 2022 greater than 10000. 23. Display cities having number of distinct departments greater than 1. 24. Display cities where maximum salary of employees joined before 2022 greater than 12000. 25. Display departments where total salary of female employees less than 15000. **Part – C:** 26. Display cities where number of male employees greater than female employees. 27. Display departments having number of cities greater than 1. 28. Display cities where total salary excluding IT department greater than 15000. 29. Display departments where average salary excluding HR employees greater than 11000. 30. Display departments where total salary of male employees greater than female employees.
Lab 10	**Implement SQL In-Built Functions for Mathematical and String Operations**
	Math functions **Part – A:** 1. Display the result of 5 multiply by 30. 2. Find out the absolute value of -25, 25, -50 and 50. 3. Find smallest integer value that is greater than or equal to 25.2, 25.7 and -25.2. 4. Find largest integer value that is smaller than or equal to 25.2, 25.7 and -25.2. 5. Find out remainder of 5 divided 2 and 5 divided by 3. 6. Find out value of 3 raised to 2nd power and 4 raised 3rd power. 7. Find out the square root of 25, 30 and 50. 8. Find out the square of 5, 15, and 25. 9. Find out the value of PI. 10. Find out round value of 157.732 for 2, 0 and -2 decimal points.

	11. Find out exponential value of 2 and 3. 12. Find out logarithm having base e of 10 and 2. 13. Find logarithm base 10 of 5 and 100 14. Find sine, cosine and tangent of 3.1415. 15. Find sign of -25, 0 and 25. 16. Generate random number using function. **String functions** **Part – A:** 1. Find the length of following. (I) NULL (II) ‘ hello ’ (III) Blank 2. Display your name in lower & upper case. 3. Display first three characters of your name. 4. Display 3rd to 10th character of your name. 5. Write a query to convert ‘abc123efg’ to ‘abcXYZefg’ & ‘abcabcabc’ to ‘ab5ab5ab5’ using REPLACE. 6. Write a query to display ASCII code for ‘a’,’A’,’z’,’Z’, 0, 9. 7. Write a query to display character based on number 97, 65,122,90,48,57. 8. Write a query to remove spaces from left of a given string ‘hello world ‘. 9. Write a query to remove spaces from right of a given string ‘ hello world ‘. 10. Write a query to display first 4 & Last 5 characters of ‘SQL Server’. 11. Write a query to convert a string ‘1234.56’ to number (Use cast and convert function). 12. Write a query to convert a float 10.58 to integer (Use cast and convert function). 13. Put 10 space before your name using function. 14. Combine two strings using + sign as well as CONCAT (). 15. Find reverse of “Darshan”. 16. Repeat your name 3 times. **Part – B: Perform following queries on EMPLOYEE table.** 17. Display FIRSTNAME and LASTNAME in lowercase and uppercase. 18. Display full name by combining FIRSTNAME and LASTNAME. 19. Display FIRSTNAME with first 3 characters only. 20. Display LASTNAME with last 2 characters only. 21. Display length of each employee’s FIRSTNAME. 22. Display FIRSTNAME after replacing ‘A’ with ‘@’. 23. Display FIRSTNAME and LASTNAME with - between them using CONCAT. **Part – C: Perform following queries on EMPLOYEE table.** 24. Display FIRSTNAME without first and last character. 25. Display FIRSTNAME after replacing vowels with '*'. 26. Display employees where combined length of FIRSTNAME and LASTNAME is greater than 10. 27. Display FIRSTNAME and its reverse. 28. Display employees whose FIRSTNAME and LASTNAME start with same character using LEFT()
Lab 11	**Implement SQL In-Built Functions for Date and Time Handling**
	Date Functions **Part – A:** 1. Write a query to display the current date & time. Label the column Today_Date. 2. Write a query to find new date after 365 day with reference to today. 3. Display the current date in a format that appears as may 5 1994 12:00AM.

4. Display the current date in a format that appears as 03 Jan 1995.
5. Display the current date in a format that appears as Jan 04, 96.
6. Write a query to find out total number of months between 31-Dec-08 and 31-Mar-09.
7. Write a query to find out total number of hours between 25-Jan-12 7:00 and 26-Jan-12 10:30.
8. Write a query to extract Day, Month, Year from given date 12-May-16.
9. Write a query that adds 5 years to current date.
10. Write a query to subtract 2 months from current date.
11. Extract month from current date using datename () and datepart () function.
12. Write a query to find out last date of current month.
13. Calculate your age in years and months.

Part – B: Perform following queries on DEPOSIT table.

14. Display all records where account date is in the year 2025.
15. Display all records where account date is in the month of March.
16. Display records where account date is after '01-Jan-2025'.
17. Display records where account date is before '01-Jan-2025'.
18. Display records where day of account date is 1.
19. Display records where month of account date is greater than 6.
20. Display records where year of account date is 2026.
21. Display number of accounts opened in each year.
22. Display number of accounts opened in each month.
23. Display maximum amount deposited in each year.

Part – C:

24. Display minimum amount deposited in each month.
25. Display total amount deposited in each year.
26. Display records where account date is between '01-Mar-2025' and '31-Dec-2025'.
27. Display records where account date is in the current year.
28. Display difference in days between today's date and account date.

Lab 12 Perform SQL queries to implement constraints

Part – A:

1. **Create STUDENT_INFO table with given constraints**

STUDENT_INFO		
COLUMN_NAME	DATATYPE	CONSTRAINT
RNO	INT	PRIMARY KEY
NAME	VARCHAR(20)	Not Null
BRANCH	VARCHAR(20)	DEFAULT 'CE'

2. Create RESULT table with given constraints

RESULT		
COLUMN_NAME	DATATYPE	CONSTRAINT
RESULTID	INT	PRIMARY KEY
SPI	DECIMAL(4,2)	CHECK (0 TO 10)
RNO	INT	FOREIGN KEY, STUDENT_INFO (RNO)

3. Insert Data into Student and Result Table

STUDENT_INFO		
RNO(PK)	NAME	BRANCH
101	RAJU	CE
102	AMIT	CE
103	SANJAY	ME
104	NEHA	EC
105	MEERA	EE
106	MAHESH	ME

RESULT		
RESULTID	SPI	RNO(FK)
11	8.8	101
12	9.2	102
13	7.6	103
14	8.2	104
15	7.0	105
16	8.9	106

4. Create DEPARTMENT table with given constraints

DEPARTMENT		
COLUMN_NAME	DATATYPE	CONSTRAINTS
DEPARTMENTID	INT	PRIMARY KEY
DEPARTMENTNAME	VARCHAR (100)	NOT NULL, UNIQUE
DEPARTMENTCODE	VARCHAR (50)	NOT NULL, UNIQUE
LOCATION	VARCHAR (50)	NOT NULL

5. Create PERSON table with given constraints

PERSON		
COLUMN_NAME	DATATYPE	CONSTRAINTS
PERSONID	INT	PRIMARY KEY
PERSONNAME	VARCHAR (100)	NOT NULL
DEPARTMENTID	INT	FOREIGN KEY, DEPARTMENT (DEPARTMENTID) NULL
SALARY	DECIMAL (8,2)	NOT NULL
JOININGDATE	DATETIME	NOT NULL
CITY	VARCHAR (100)	NOT NULL

6. Insert Data into PERSON and DEPARTMENT Table

DEPARTMENTID	DEPARTMENTNAME	DEPARTMENTCODE	LOCATION
1	ADMIN	ADM	A-BLOCK
2	COMPUTER	CE	C-BLOCK
3	CIVIL	CI	G-BLOCK
4	ELECTRICAL	EE	E-BLOCK
5	MECHANICAL	ME	B-BLOCK
6	CHEMICAL	CH	D-BLOCK

PERSONID	PERSONNAME	DEPARTMENTID	SALARY	JOININGDATE	CITY
101	RAHUL TRIPATHI	2	56000	01-01-2000	RAJKOT
102	HARDIK PANDYA	3	18000	25-09-2001	AHMEDABAD
103	BHAVIN KANANI	4	25000	14-05-2000	BARODA
104	BHOOMI VAISHNAV	1	39000	08-02-2005	RAJKOT
105	ROHIT TOPIYA	2	17000	23-07-2001	JAMNAGAR
106	PRIYA MENPARA	NULL	9000	18-10-2000	AHMEDABAD
107	NEHA SHARMA	2	34000	25-12-2002	RAJKOT
108	NAYAN GOSWAMI	3	25000	01-07-2001	RAJKOT
109	MEHUL BHUNDIYA	4	13500	09-01-2005	BARODA
110	MOHIT MARU	5	14000	25-05-2000	JAMNAGAR

Part – B:

7. **Create** PUBLISHER **table with given constraints**

PUBLISHER		
COLUMN NAME	**DATA TYPE**	**CONSTRAINTS**
PUBLISHERID	INT	PRIMARY KEY
PUBLISHERNAME	VARCHAR(100)	NOT NULL, UNIQUE
CITY	VARCHAR(50)	NOT NULL

8. **Create** AUTHOR **table with given constraints**

AUTHOR		
COLUMN NAME	**DATA TYPE**	**CONSTRAINTS**
AUTHORID	INT	PRIMARY KEY
AUTHORNAME	VARCHAR(100)	NOT NULL
COUNTRY	VARCHAR(50)	NULL

9. **Create** BOOK **table with given constraints**

BOOK		
COLUMN NAME	**DATA TYPE**	**CONSTRAINTS**
BOOKID	INT	PRIMARY KEY
TITLE	VARCHAR(200)	NOT NULL
AUTHORID	INT	FOREIGN KEY, AUTHOR(AUTHORID), NOT NULL
PUBLISHERID	INT	FOREIGN KEY, PUBLISHER(PUBLISHERID), NOT NULL
PRICE	DECIMAL(8,2)	NOT NULL
PUBLICATIONYEAR	INT	NOT NULL

10. **Insert Data into PUBLISHER, AUTHOR and BOOK Table**

PUBLISHERID	PUBLISHERNAME	CITY
1	RUPA PUBLICATIONS	NEW DELHI
2	PENGUIN INDIA	GURUGRAM
3	HARPERCOLLINS INDIA	NOIDA
4	ALEPH BOOK COMPANY	NEW DELHI

AUTHORID	AUTHORNAME	COUNTRY
1	CHETAN BHAGAT	INDIA
2	ARUNDHATI ROY	INDIA
3	AMISH TRIPATHI	INDIA
4	RUSKIN BOND	INDIA
5	JHUMPA LAHIRI	INDIA
6	PAULO COELHO	BRAZIL
7	SUDHA MURTY	INDIA

BOOKID	TITLE	AUTHORID	PUBLISHERID	PRICE	PUBLICATIONYEAR
101	FIVE POINT SOMEONE	1	1	250.00	2004
102	THE GOD OF SMALL THINGS	2	2	350.00	1997
103	IMMORTALS OF MELUHA	3	3	300.00	2010
104	THE BLUE UMBRELLA	4	1	180.00	1980
105	THE LOWLAND	5	2	400.00	2013
106	REVOLUTION 2020	1	1	275.00	2011
107	SITA: WARRIOR OF MITHILA	3	3	320.00	2017
108	THE ROOM ON THE ROOF	4	4	200.00	1956

11. Create STADIUM table with given constraints

STADIUM		
COLUMN NAME	DATA TYPE	CONSTRAINTS
Stadium_id	INT	PRIMARY KEY
Stadium_name	VARCHAR(200)	NOT NULL, UNIQUE
Stadium_capacity	INT	NOT NULL
Stadium_city	VARCHAR(200)	NOT NULL

12. Create TEAM table with given constraints

TEAM		
COLUMN NAME	DATA TYPE	CONSTRAINTS
TEAM_ID	INT	PRIMARY KEY
TEAM_NAME	VARCHAR(200)	NOT NULL, UNIQUE
TEAM_COACH	VARCHAR(200)	NOT NULL
TEAM_WINS	INT	NOT NULL
TEAM_TOTAL_MATCHES	INT	NULL
HOME_STADIUM_ID	INT	FOREIGN KEY STADIUM(STADIUM_ID), NULL

13. Create PLAYER table with given constraints

PLAYER		
COLUMN NAME	DATA TYPE	CONSTRAINTS
PLAYER_ID	INT	PRIMARY KEY
PLAYER_FIRST_NAME	VARCHAR(200)	NOT NULL
PLAYER_LAST_NAME	VARCHAR(200)	NOT NULL
TEAM_ID	INT	FOREIGN KEY TEAM(TEAM_ID), NULL
PLAYER_ROLE	VARCHAR(200)	NULL
PLAYER_JERSEY_NUMBER	INT	NOT NULL
PLAYER_MATCHES_PLAYED	INT	NULL

14. Insert Data into STADIUM and TEAM Table

STADIUM			
Stadium_id	Stadium_name	Stadium_capacity	Stadium_city
101	Wankhede Stadium	55000	Mumbai
102	Chepauk Stadium	50000	Chennai
103	Chinnaswamy Stadium	45000	Bangalore
104	Eden Gardens	68000	Kolkata
105	Sawai Mansingh Stadium	30000	Jaipur
106	Arun Jaitley Stadium	42000	Delhi
107	Bindra Stadium	38000	Mohali
108	Rajiv Gandhi Stadium	55000	Hyderabad

TEAM					
TEAM_ID	TEAM_NAME	TEAM_COACH	TEAM_WINS	TEAM_TOTAL _MATCHES	HOME_ STADIUM_ID
1	Mumbai Indians	Mark Boucher	12	14	101
2	Chennai Super Kings	Stephen Fleming	10	14	102
3	Royal Challengers Bangalore	Faf du Plessis	9	14	103
4	Kolkata Knight Riders	Gautam Gambhir	11	14	104
5	Rajasthan Royals	Rahul Dravid	8	14	105
6	Delhi Capitals	Ricky Ponting	7	14	106
7	Punjab Kings	Anil Kumble	6	14	107
8	Sunrisers Hyderabad	Brian Lara	9	14	108

15. **Insert Data into PLAYER Table**

PLAYER

PLAYER_ID	PLAYER_FIRST_NAME	PLAYER_LAST_NAME	TEAM_ID	PLAYER_ROLE	PLAYER_JERSEY_NUMBER	PLAYER_MATCHES_PLAYED
201	Virat	Kohli	3	Batsman	18	25
202	Rohit	Sharma	1	Batsman	45	28
203	Jasprit	Bumrah	1	Bowler	93	26
204	MS	Dhoni	2	Wicketkeeper	7	30
205	Ravindra	Jadeja	2	All-rounder	8	27
206	Andre	Russell	4	All-rounder	12	24
207	Sanju	Samson	5	Batsman	11	23
208	Yuzvendra	Chahal	5	Bowler	3	22
209	Glenn	Maxwell	3	All-rounder	32	21
210	Sunil	Narine	4	Bowler	74	29
211	David	Warner	6	Batsman	31	26
212	Rishabh	Pant	6	Wicketkeeper	17	24
213	Kagiso	Rabada	6	Bowler	25	23
214	Shikhar	Dhawan	7	Batsman	42	27
215	Liam	Livingstone	7	All-rounder	23	22
216	Arshdeep	Singh	7	Bowler	2	21
217	Aiden	Markram	8	Batsman	4	23
218	Bhuvneshwar	Kumar	8	Bowler	15	28
219	Rahul	Tripathi	8	Batsman	52	24
220	Abdul	Samad	8	All-rounder	11	29

Lab 13 Implement SQL Joins to Combine Multiple Tables

From the table STUDENT_INFO and RESULT perform the following queries:

Part – A:

1. Combine information from Student and Result table using cross join (Cartesian product).
2. Perform inner join on Student and Result tables.
3. Perform the left outer join on Student and Result tables.
4. Perform the right outer join on Student and Result tables.
5. Perform the full outer join on Student and Result tables.
6. Display Rno, Name, Branch and SPI of all students.
7. Display Rno, Name, Branch and SPI of CE branch students only.
8. Display Rno, Name, Branch and SPI of students other than EC branch.
9. Display Rno, Name and SPI of students whose SPI is greater than 8.
10. Display Rno, Name and Branch of students whose SPI is less than 8.
11. Display average result of each branch.
12. Display average result of CE and ME branch.
13. Display maximum and minimum SPI of each branch.
14. Display branch-wise student count in descending order.
15. Display branch-wise total SPI of students.

Part – B:

16. Display branch-wise number of students having SPI greater than 8.
17. Display branch-wise number of students having SPI less than 8.

	18. Display branch-wise average SPI greater than 7. 19. Display branches having more than 1 students. 20. Display branches where maximum SPI is greater than 9. **Part – C:** 21. Display average result of each branch and sort them in ascending order by SPI. 22. Display highest SPI from each branch and sort them in descending order. 23. Display average result of each branch and sort them in ascending order by SPI. 24. Display highest SPI from each branch and sort them in descending order. 25. Display branches where difference between max and min SPI is greater than 1.
Lab 14	**Implement Complex SQL Joins with Multiple Tables and Conditions**
	From the table DEPARTMENT and PERSON perform the following queries: **Part – A:** 1. Combine information from Person and Department table using cross join or Cartesian product. 2. Find all persons with their department name 3. Find all persons with their department name & code. 4. Find all persons with their department code and location. 5. Find the detail of the person who belongs to Mechanical department. 6. Final person's name, department code and salary who lives in Ahmedabad city. 7. Find the person's name whose department is in C-Block. 8. Retrieve person name, salary & department name who belongs to Jamnagar city. 9. Retrieve person's detail who joined the Civil department after 1-Aug-2001. 10. Display all the person's name with the department whose joining date difference with the current date is more than 25 years. 11. Find department wise person counts. 12. Give department wise maximum & minimum salary with department name. 13. Find city wise total, average, maximum and minimum salary. 14. Find the average salary of a person who belongs to Ahmedabad city. 15. Produce Output Like: <PersonName> lives in <City> and works in <DepartmentName> Department. (In single column) **Part – B:** 16. Produce Output Like: <PersonName> earns <Salary> from <DepartmentName> department monthly. (In single column) 17. Find city & department wise total, average & maximum salaries. 18. Find all persons who do not belong to any department. 19. Find all departments whose total salary is exceeding 100000. **Part – C:** 20. List all departments who have no person. 21. List out department names in which more than two persons are working. 22. Give a 10% increment in the computer department employee's salary. (Use Update)
Lab 15	**Implement Advanced SQL Joins (Level-1)**
	From the table PUBLISHER, AUTHOR and BOOK perform the following queries: **Part – A:** 1. List all books with their authors. 2. List all books with their publishers.

3. List all books with their authors and publishers.
4. List all books published after 2010 with their authors and publisher and price.
5. List all authors and the number of books they have written.
6. List all publishers and the total price of books they have published.
7. List authors who have not written any books.
8. Display the total number of books written by each author along with the average price of their books.
9. lists each publisher along with the total number of books they have published, sorted from highest to lowest.
10. Display number of books published each year.

Part – B:

EMPLOYEE_MASTER		
EmployeeNo	Name	ManagerNo
E01	Tarun	NULL
E02	Rohan	E02
E03	Priya	E01
E04	Milan	E03
E05	Jay	E01
E06	Anjana	E04

11. List the publishers whose total book prices exceed 500, ordered by the total price.
12. List most expensive book for each author, sort it with the highest price.
13. Display publisher name and difference between maximum and minimum book price.
14. List publisher name and total price of books published each year.
15. Display author name and total price of books sorted by highest total price.

From the above table EMPLOYEE_MASTER perform the following queries:

Part – C:

16. Retrieve the names of employee along with their manager's name from the Employee table.
17. Display employees who are managers.
18. Display number of employees working under each manager.
19. Display the employee's name along with their manager's name and senior manager name.
20. Display managers and count of employees under them in descending order.

Lab 16 Implement Advanced SQL Joins (Level-2)

From the table STADIUM, TEAM and PLAYER perform the following queries:

Part – A:

1. Display players who belong to teams located in 'Mumbai'.
2. Display all teams and players.
3. Display players along with team wins and stadium city.
4. Display team name and number of players in each team.
5. Display team name, coach, and number of bowlers in each team.
6. Display team name with count of batsmen, bowlers, and all-rounders.
7. Display stadiums where teams have won more than 10 matches.
8. Display team name and number of players whose matches played is greater than 25.
9. Display team name and total number of players having jersey number greater than 30.
10. Display team name and total matches played by its players.

	Part – B: 11. Display stadium city and total number of teams in each city. 12. Display team name and average matches played by players in each team. 13. Display team name and maximum matches played by any player in each team. 14. Display team name and minimum matches played by any player in each team. 15. Display stadium name and total number of players playing under teams of that stadium. **Part – C:** 16. Display teams having more all-rounders than bowlers. 17. Display teams where difference between max and min player matches is greater than 5. 18. Display stadium city and total wins of teams in that city. 19. Display team name and total number of players for each role (grouped by role). 20. Display team name and total number of players whose name starts with 'A'
Lab 17	**Implement SQL View**
	From the table EMPLOYEE perform the following queries: **Part – A:** 1. Create a view Employee_All with all columns. 2. Create a view Employee_NameDeptSalary having columns FirstName, Department and Salary. 3. Create a view Employee_Basic having columns EID, FirstName and City. 4. Create a view IT_Employees that displays IT department data only. 5. Create a view HR_Employees that displays HR department data only. 6. Create a view Employee_2026 that displays employees joined in 2026 only. 7. Create a view Patel_Employees that displays employees whose last name is PATEL. 8. Create a view High_Salary_Emp having all columns but employees whose salary is more than 12000. 9. Create a view that displays information of all employees whose salary is above 14000. 10. Create a view that displays employees having salary below 10000. 11. Create a view Server_Dept that displays Server department employees only. 12. Insert a new record into Employee_Basic view. (111, MEET, SURAT) 13. Update the department of DEEP from ADMIN to IT in Employee_NameDeptSalary view. 14. Delete an employee whose EID is 107 from Employee_Basic view. 15. Drop IT_Employees view from the database. **Part – B:** 16. Create a view Admin_Employees that displays ADMIN department employees only. 17. Create a view Female_Employees that displays female employee data only. 18. Create a view Male_Employees that displays male employee data only. 19. Create a view Rajkot_Employees that displays employees from Rajkot city only. 20. Create a view Ahmedabad_Employees that displays employees from Ahmedabad city only. 21. Create a view Salary_Between that displays employees whose salary is between 10000 and 14000. 22. Create a view Recent_Employees that displays employees joined after 2023. 23. Create a view Old_Employees that displays employees joined before 2023. 24. Create a view Employees_Start_R that displays employees whose first name starts with R. 25. Create a view Employees_End_A that displays employees whose first name ends with A. **Part – C:** 26. Create a view Employees_NameContains_H that displays employees whose first name contains H. 27. Create a view for the employees whose first name contains vowels. 28. Create a view FourLetter_Name having EID, FirstName and Department columns in which FirstName consists of four letters. 29. Create a view for the employees whose name starts with M and ends with N. 30. Create a view Transport_Dept that displays Transport department employees only.

Lab 18 Perform SQL Queries Using Subqueries

From the table STUDENT perform the following queries:

Part – A:

1. Display the details of students whose SPI is greater than the average SPI.
2. Display the names of students whose SPI is less than the average SPI.
3. Display the student details who has the highest SPI.
4. Display the student details who has the lowest SPI.
5. Display the students whose SPI is greater than SPI of student DHARMIK.
6. Display the students whose SPI is less than SPI of student RIYA.
7. Display the students who belong to the same branch as KRUNAL.
8. Display the students whose branch is different from HETVI.
9. Display the second highest SPI from RESULT table.
10. Display the second lowest SPI from RESULT table.
11. Display the names of students whose SPI is above branch-wise average SPI.
12. Display the branch having maximum average SPI.
13. Display the branch having minimum average SPI.

From the table STUDENT_INFO and RESULT perform the following queries:

Part – B:

14. Display the students whose SPI is greater than all students of ME branch.
15. Display the students whose SPI is less than any student of ME branch.
16. Display the student details whose SPI is not equal to any SPI of EC branch students.
17. Display the names of students who scored higher SPI than student of RNO 103.
18. Display the students whose SPI is greater than average SPI of their own branch.
19. Display the students whose SPI is greater than the average SPI of CE branch but greater than the maximum SPI of ME branch.
20. Display the branch names whose average SPI is greater than the overall average SPI.
21. Display the students who have maximum SPI in their respective branch.
22. Display the students whose SPI is greater than their average SPI of their branch and greater than overall average SPI.

Part – C:

23. Display the students whose SPI is greater than at least one student of every branch.
24. Display the students whose SPI is less than all students of CE branch.
25. Display the branch that contains the student with highest SPI.
26. Display the students whose SPI is less than the SPI of every student in CE branch and greater than every student in ME branch.

Lab 19 Perform SQL Queries Using Set Operators

Create below tables as per following data.

ONLINE_CUSTOMER

CID	Name	CITY
101	RAHUL	RAJKOT
103	PTIYA	SURAT
106	DHARMIK	SURAT
107	MEHUL	MORBI

OFFLINE_CUSTOMER

CID	Name	CITY
102	AMIT	AHMEDABAD
104	HETVI	BARODA
105	RIYA	RAJKOT
106	DHARMIK	SURAT

CUSTOMER

CID	Name	CITY	MEMBERSHIP
101	RAHUL	RAJKOT	GOLD
102	AMIT	AHMEDABAD	SILVER
103	PTIYA	SURAT	GOLD
104	HETVI	BARODA	PLATINUM
105	RIYA	RAJKOT	SILVER
106	DHARMIK	SURAT	GOLD
108	ISHITA	RAJOT	GOLD

From the above table perform the following queries:

Part – A:

1. Display all unique customer names from ONLINE_CUSTOMER and OFFLINE_CUSTOMER.
2. Display all unique customer cities from ONLINE_CUSTOMER and OFFLINE_CUSTOMER.
3. Display duplicate customer names from both tables ONLINE_CUSTOMER and OFFLINE_CUSTOMER.
4. Display unique customer IDs from both tables.
5. Display all GOLD membership customer names and online customer names using UNION.
6. Display all customer names from CUSTOMER and ONLINE_CUSTOMER using UNION.
7. Display all cities from CUSTOMER and OFFLINE_CUSTOMER using UNION.
8. Display all customer IDs from CUSTOMER and OFFLINE_CUSTOMER using UNION ALL.
9. Display all customer names from RAJKOT city using UNION.
10. Display all customer names starting with R from both tables using UNION.
11. Display customers who are both ONLINE and OFFLINE.
12. Display customer IDs existing in both ONLINE_CUSTOMER and OFFLINE_CUSTOMER.
13. Display customer names existing only in ONLINE_CUSTOMER.
14. Display customer names existing only in OFFLINE_CUSTOMER.
15. Display cities available in both ONLINE_CUSTOMER and OFFLINE_CUSTOMER.

Part – B:

16. Display customer names who are GOLD members and online customers.
17. Display customer names who are PLATINUM members but not online customers.
18. Display customer IDs who are online customers but not offline customers.
19. Display customer IDs who are offline customers but not online customers.
20. Display customers who are present in CUSTOMER table but not in ONLINE_CUSTOMER.
21. Display customers who are both GOLD members and available in OFFLINE_CUSTOMER.
22. Display all distinct cities from all three tables.
23. Display customers who are online customers and belong to SURAT city using INTERSECT.
24. Display customers who are offline customers but not from RAJKOT city using MINUS.
25. Display customers who are GOLD members, online customers, and from SURAT city using INTERSECT.

Part – C:

26. Display customer names who are present in at least two tables.
27. Display all customer names from CUSTOMER table excluding customers available in both ONLINE_CUSTOMER and OFFLINE_CUSTOMER.
28. Display customer names who are present in both ONLINE_CUSTOMER and OFFLINE_CUSTOMER but not PLATINIUM members.
29. Display customer names who are either online or offline but not both.
30. Display customer IDs present in CUSTOMER table but missing from both ONLINE_CUSTOMER and OFFLINE_CUSTOMER.

Lab 20	**Implement Window Functions for Advanced Data Analysis**
	From the table STUDENT perform the following queries: **Part – A:** 1. Display rank of students based on SPI. 2. Display dense rank of students based on SPI. 3. Display sequential number for each student record. 4. Display branch-wise rank of students. 5. Display branch-wise dense ranking of students. 6. Display branch-wise sequential numbering of students. 7. Display SNAME, Current SPI, Previous SPI and SPI Difference with previous student in ascending order of SPI. 8. Display SNAME, Current SPI, Next SPI and SPI Difference with next student in descending order of SPI. 9. Display top 3 students based on SPI. 10. Display top 2 students from each branch. **Part – B:** 11. Display 5th highest SPI. 12. Display 6th highest SPI. 13. Display students having same ranking. 14. Display SNAME, Previous SPI, Current SPI and Next SPI based on ascending order of SPI. 15. Display topper of each branch. **Part – C:** 16. Display students whose SPI is greater than the previous student and less than the next student. 17. Display branch-wise second topper students. 18. Display students whose rank and dense rank are different. 19. Display consecutive students having same branch ordered by SPI. 20. Display students whose SPI difference with previous student is maximum.
Lab 21	**Implement Intermediate Common Table Expressions (CTE) for Query Simplification**
	From the table STUDENT perform the following queries: **Part – A:** 1. Display all students whose SPI is greater than 8. 2. Display average SPI of all students. 3. Display total number of students in each branch. 4. Display students who belong to RAJKOT city. 5. Find branch names that appear more than once. 6. Display row number for each student. 7. Display top 3 students based on SPI. 8. Display students having maximum SPI. 9. Display students having minimum SPI. 10. Display branch -wise rank of students. **Part – B:** 11. Display students SPI average belonging to Computer branch. 12. Display students whose SPI is greater than average SPI of his/her branch. 13. Display branch having more than 2 students. 14. Display branches having average SPI between 7 and 9

15. Display students whose SPI is lower than overall average SPI.

Part – C:

16. Display branches having exactly one student.
17. Display branch having highest average SPI.
18. Display branch having lowest average SPI.
19. Display students whose SPI is lower than branch average SPI.
20. Display branches having maximum number of students.

Lab 22 Implement Advance Common Table Expressions (CTE) for Query Simplification

From the table CUSTOMER perform the following queries:

CUSTOMER_ALL

ORDERID	CNAME	PRODUCT	CATEGORY	AMOUNT	ORDERYEAR	CITY
101	RAHUL	LAPTOP	ELECTRONICS	65000	2024	RAJKOT
102	PRIYA	MOBILE	ELECTRONICS	25000	2023	SURAT
103	AMIT	TABLE	FURNITURE	12000	2022	AHMEDABAD
104	NEHA	CHAIR	FURNITURE	8000	2024	BARODA
105	VISHAL	TV	ELECTRONICS	45000	2025	MORBI
106	RIYA	SOFA	FURNITURE	30000	2023	SURAT
107	MEHUL	AC	ELECTRONICS	40000	2022	RAJKOT
108	KRUNAL	BED	FURNITURE	40000	2025	JAMNAGAR

Part – A:

1. Display top 3 highest amount orders.
2. Display second highest order amount.
3. Display customers whose order amount is greater than category average amount.
4. Display categories having average amount greater than 30000.
5. Display highest amount order from each category.
6. Display lowest amount order from each category.
7. Display categories having more than 3 orders.
8. Display city-wise total order amount.
9. Display category having highest average order amount.
10. Display cumulative order amount in ascending order of amount.

Part – B:

11. Display category-wise top 2 highest amount orders.
12. Display customers whose amount is closest to category average amount.
13. Display previous, current and next order amount together.
14. Display customers whose amount is greater than previous customer's amount.
15. Display customers whose rank and dense rank are different.

Part – C:

16. Display orders whose amount is neither highest nor lowest in their category.
17. Display category-wise difference between highest and lowest amount.
18. Display customers whose amount is greater than all FURNITURE category orders.
19. Display categories where all orders are above 10000.
20. Display customers whose amount difference from category topper is minimum.

Lab 23 — Implement Stored Procedures for Reusable SQL Operations

From the table STUDENT perform the following queries:

Part – A:

1. **INSERT Procedures:** Create stored procedures to insert records into STUDENT tables (SP_INSERT_STUDENT)

STDID	SNAME	CITY	SPI	BRANCH
115	PUSHTI	RAJKOT	9.48	COMPUTER
116	NIKUNJ	SURAT	8.80	CHEMICAL

2. **INSERT Procedures:** Create stored procedures to insert records into DEPOSIT tables (SP_INSERT_DEPOSIT)

ACTNO	CNAME	BNAME	AMOUNT	ADATE
118	HEMENT	BEDI	16000	05-05-2025
119	RAVI	MAVDI	24000	09-07-2024

3. **UPDATE Procedures:** Create stored procedure SP_UPDATE_STUDENT to update Branch in STUDENT table. (Update using studentID)

STDID	BRANCH
115	ELECTRICAL
116	MECHANICAL

4. **DELETE Procedures:** Create stored procedure SP_DELETE_STUDENT to delete records from STUDENT where Student Name is **RAVI**.
5. **SELECT BY PRIMARY KEY:** Create stored procedures to select records by primary key (SP_SELECT_STUDENT_BY_ID) from Student table. (Display All Columns)
6. Create a stored procedure that shows details of the first 5 students ordered by SPI (Highest First).

From the table EMPLOYEE perform the following queries:

Part – B:

7. Create a stored procedure which displays all employee details.
8. Create a stored procedure that takes department name as input and returns all the employee in that department.

Part – C:

9. Create a stored procedure which displays department-wise maximum, minimum, and average salary of employee.
10. Create a stored procedure that accepts department name as parameter and returns total salary of their department.

Lab 24 — Implement Advanced Stored Procedures with Conditions and Logic

From the table EMPLOYEE perform the following queries:

Part – A:

1. Create a stored procedure to generate department-wise salary statistics like total salary, average salary, minimum salary, and maximum salary. (User enter only department name)
2. Create a stored procedure that accepts a joining year and displays employees who joined that year.

	3. Create a stored procedure for dynamic employee search using parameters (User may enter partial city name). 4. Create a stored procedure that accepts a salary amount and displays employees earning more than the entered salary. 5. Create a stored procedure to display top N highest paid employees from each department (Value of N is entered by user). 6. Create a stored procedure to increase salary department-wise by a given percentage. (User Enter Department Name and %, e.g. Computer 10). 7. Create a stored procedure to display employees having experience greater than or equal to the entered years. 8. Create a stored procedure that accepts a number as input and displays details of the last N employees who joined the organization. **From the table AUTHOR, PUBLISHER and BOOK perform the following queries:** **Part – B:** 9. Create a stored procedure that accepts an author name and displays all books written by that author. 10. Create a stored procedure that accepts a publication year and displays books published after that year. 11. Create a stored procedure that accepts a country name and displays all authors from that country with their books. 12. Create a stored procedure that accepts a number as input and displays the top N most expensive books with author and publisher details. **Part – C:** 13. Create a stored procedure that accepts a publisher name and displays the total number of books published by that publisher. 14. Create a stored procedure that accepts a price range (Min Price Max Price) and displays books whose prices fall within that range. 15. Create a stored procedure that accepts an author ID and deletes all books written by that author.
Lab 25	**Implement User Defined Functions (UDF) in SQL (Scalar Function)**
	Part – A: 1. Implement scalar function to return "Welcome to DBMS Lab". 2. Implement scalar function to calculate simple interest. 3. Implement scalar function to find difference in days between two dates. 4. Implement scalar function to check whether number is odd or even. 5. Implement scalar function to print numbers from 1 to N. **Part – B:** 6. Implement scalar function to calculate factorial of given number. 7. Implement scalar function to check palindrome number. 8. Implement scalar function to find maximum of three numbers. 9. Implement scalar function to calculate square and cube of a number. **From the table EMPLOYEE perform the following queries:** **Part – C:** 10. Implement scalar function to return employee full details using EID. 11. Implement scalar function to return highest salary from a given department. 12. Implement scalar function to count total employees in EMPLOYEE table. 13. Implement scalar function to find total experience of employee using JoiningYear. 14. Implement scalar function to return total number of employees in a given department. 15. Implement scalar function to count total employees from a given city.

Lab 26	**Implement User Defined Functions (UDF) in SQL (Table Valued Function)**
	From the table STUDENT perform the following queries: **Part – A:** 1. Create a table valued function to display all student records. 2. Create a table valued function that accepts CITY and returns all students from that city. 3. Create a table valued function that accepts BRANCH and returns all students of that branch. 4. Create a table valued function that accepts SPI and returns students whose SPI is greater than entered SPI. 5. Create a table valued function that accepts MIN_SPI and MAX_SPI and returns students whose SPI lies between given range. **Part – B:** 6. Create a table valued function that accepts STDID and returns details of that student. 7. Create a table valued function that accepts CITY and returns students whose SPI is greater than 7 from that city. 8. Create a table valued function that accepts BRANCH and returns students whose SPI is less than 8 from that branch. 9. Create a table valued function that accepts TOPN and returns top N students based on SPI. 10. Create a table valued function that accepts BRANCH and returns highest SPI student from that branch. **Part – C:** 11. Create a table valued function that accepts CITY and returns total students from that city. 12. Create a table valued function that accepts BRANCH and returns students ordered by SPI in descending order. 13. Create a table valued function that accepts CITY and returns top 3 student from that city based on SPI. 14. Create a table valued function that accepts STDID and returns student rank based on SPI (RANK). 15. Create a table valued function that accepts BRANCH and returns students having second highest SPI from that branch.
Lab 27	**Implement Cursors for Row-by-Row Data Processing**
	From the table EMPLOYEE perform the following queries: **Part – A:** 1. Create a cursor Employee_Cursor to fetch all rows from EMPLOYEE table and display them. 2. Create a cursor to display all female employees from EMPLOYEE table. 3. Create a cursor Employee_Cursor_Fetch to fetch records in form of EID_FirstName_LastName. (Example: 101_Hetvi_Patel) 4. Create a cursor to find and display all employees with Salary greater than 12000. 5. Create a cursor to display all employees who joined in year 2022 or later. 6. Create a cursor to fetch Employee Name with Department Name. (Example: Raj Mehta works in IT Department) 7. Create a cursor to update CITY as 'AHMEDABAD' for employees whose CITY value is NULL. 8. Create a cursor to display Employee Name with City Name. (Example: Deep Patel lives in Rajkot) 9. Create a cursor to delete employees whose Salary is less than 5000. 10. Create a cursor to display employees department-wise. **Part – B:** 11. Create a cursor to count total employees from each department. 12. Create a cursor to display employees whose CITY starts with letter 'R'. 13. Create a cursor to display top 3 highest salary employees from EMPLOYEE table. 14. Create a cursor to calculate total salary department-wise. (Example: IT Department total salary = 33000) 15. Create a cursor to calculate average salary city-wise.

	Part – C: 16. Create a cursor to display employee experience using JOININGYEAR. (Example: Raj Mehta has experience = 4 years) 17. Create a cursor to display employee full name with annual salary. (Example: Hetvi Patel annual salary = 144000) 18. Create a cursor to calculate total male and female employees separately. 19. Create a cursor to display employees whose salary is greater than department average salary. 20. Create a cursor to transfer all employees from ADMIN department to HR department.
Lab 28	**Implement AFTER Triggers for Automatic Execution After Data Operations**
	EMPLOYEE_LOG (LOGID, EID, OLDVALUE, NEWVALUE, FIELDNAME, OPERATIONTYPE, LOGDATE) **From the table EMPLOYEE perform the following queries:** **Part – A:** 1. Create trigger for printing message after employee record insertion. 2. Create trigger for printing message after employee record update. 3. Create trigger for printing message after employee record deletion. 4. Create trigger for printing message after employee salary increment. 5. Create trigger for automatically converting CITY names into uppercase during insertion. **Part – B:** 6. Create trigger for updating employee city and printing old city and new city name. 7. Create trigger for automatically setting CITY as 'RAJKOT' if no city value is entered during employee insertion. 8. Create trigger for automatically adding current year in JOININGYEAR if no value is entered. 9. Create trigger for printing employee full name after new employee insertion. 10. Create trigger for automatically assigning department as 'GENERAL' if DEPARTMENT value is NULL. **Part – C:** 11. Create trigger for storing updated employee details such as EID, old salary, new salary, old department, new department, and update date into EMPLOYEE_UPDATE_LOG table. 12. Create trigger for storing newly inserted employee details with insertion date into EMPLOYEE_INSERT_LOG table. 13. Create trigger for storing old and new FIRSTNAME values after employee name update into NAME_CHANGE_LOG table. 14. Create trigger for storing old city and new city details into CITY_UPDATE_LOG table after city update. 15. Implement INSTEAD OF INSERT trigger on EMPLOYEE table to automatically remove extra spaces from FIRSTNAME and LASTNAME before insertion.
Lab 29	**Implement INSTEAD OF Triggers**
	EMPLOYEE_LOG (LOGID, EID, OLDVALUE, NEWVALUE, FIELDNAME, OPERATIONTYPE, LOGDATE) **From the table EMPLOYEE perform the following queries:** **Part – A:** 1. Create trigger for preventing removal of employees from the table. 2. Create INSTEAD OF DELETE trigger to prevent deletion of employee records and store deleted data into EMPLOYEE_LOG table. 3. Create INSTEAD OF trigger to log all operations on EMPLOYEE table (INSERT/UPDATE/DELETE) into EMPLOYEE_LOG table.

	4. Create trigger to block employees from updating their JOININGYEAR and print message 'Employees are not allowed to update their joining year'. 5. Create trigger for preventing duplicate employee records having same FIRSTNAME, LASTNAME, and CITY. **Part – B:** 6. Create INSTEAD OF INSERT trigger to prevent insertion of duplicate EID in EMPLOYEE table. 7. Create INSTEAD OF UPDATE trigger to maintain complete employee update history into EMPLOYEE_LOG table. 8. Create INSTEAD OF UPDATE trigger to prevent changing employee EID once record is created. 9. Create INSTEAD OF INSERT trigger to block insertion of employees with NULL department name. 10. Create INSTEAD OF UPDATE trigger to prevent updating GENDER column after employee registration. **STUDENT_LOG (LOGID, STDID, SNAME, OLDVALUE, NEWVALUE, FIELDNAME, OPERATIONTYPE, LOGDATE)** **From the table STUDENT perform the following queries:** **Part – C:** 11. Create INSTEAD OF INSERT trigger to prevent insertion of students whose SPI is greater than 10 or less than 0. 12. Create INSTEAD OF UPDATE trigger to block students from changing their BRANCH after admission. 13. Create INSTEAD OF DELETE trigger to move deleted student records into STUDENT_LOG table instead of permanent deletion. 14. Create INSTEAD OF UPDATE trigger to prevent updating student SPI. 15. Create INSTEAD OF UPDATE trigger to store student branch transfer history into STUDENT_LOG table.
Lab 30	**Implement Exception Handling Using TRY...CATCH, Throw and Raise Error in SQL**
	Part – A: 1. Handle Divide by Zero Error and Print message like: Error occurs that is - Divide by zero error. 2. Try to convert string to integer and handle the error using try…catch block. 3. Create a procedure that prints the sum of two numbers: take both numbers as integer & handle exception with all error functions if any one enters string value in numbers otherwise print result. 4. Handle a Primary Key Violation while inserting data into STUDENT_INFO table and print the error details such as the error message, error number, severity, and state. 5. Throw custom exception using stored procedure which accepts RNO as input & that throws Error like no RNO is available in database. **Part – B** 6. Create a stored procedure to update employee SALARY and throw custom exception if salary is negative or zero (Use EMPLOYEE Table). 7. Handle a Foreign Key Violation while inserting data into RESULT table and print appropriate error message (Use RESULT Table). 8. Handle Invalid Date Format while inserting data into DEPOSIT table. 9. Create a stored procedure that validates gender column and throws error if value is other than male or female (Use EMPLOYEE Table). 10. Create a stored procedure that accepts joiningyear and throws custom exception if entered year is greater than current year (Use EMPLOYEE Table). **Part – C** 10. Create a stored procedure to delete employee record and handle exception if employee does not exist. 11. Create a stored procedure that throws custom exception if department name is NULL during insertion.